EF Core: Include(), ThenInclude() & JOIN Types

Concise lesson notes: loading related data, relationship paths, and understanding LEFT JOIN vs INNER JOIN.

1. Include() vs ThenInclude()

Include() loads a direct navigation property. **ThenInclude()** continues from the included relationship to a deeper related entity.

Mental model: Student → Enrollments → Course → Instructor

```
var students = context.Students
    .Include(s => s.Enrollments)
    .ThenInclude(e => e.Course)
    .ThenInclude(c => c.Instructor)
    .ToList();
```

• **Include:** Student → Enrollments (first relationship level).

• **ThenInclude:** Enrollment → Course, then Course → Instructor (deeper levels).

• **New Include:** starts a separate branch, such as Student → StudentProfile.

Separate branches

```
var students = context.Students
    .Include(s => s.Enrollments)
        .ThenInclude(e => e.Course)
    .Include(s => s.StudentProfile)
    .ToList();
```

This loads two paths: Students → Enrollments → Course, and Students → StudentProfile.

2. LEFT JOIN vs INNER JOIN — explanation of the image

The image illustrates two different relationships in one query. JOIN type is affected by relationship optionality and query translation; it is not a universal rule that every Include is LEFT JOIN or every ThenInclude is INNER JOIN.

Relationship	JOIN shown	Reason
Students → Enrollments	**LEFT JOIN**	A student may have zero enrollments. Keep the student even when no enrollment matches.
Enrollments → Course	**INNER JOIN**	The image models each enrollment as requiring one course, so only matching courses are returned for that relationship.

Example shown: Ahmed has two enrollments, so the joined result has rows for C# and SQL. Sara has one enrollment, so she appears once. Omar has no enrollment, but remains because Students → Enrollments uses LEFT JOIN; columns from the unmatched related data can be NULL.

Important nuance: required relationships often allow EF Core to use INNER JOIN, while optional relationships often need LEFT JOIN to preserve the parent row. Exact SQL can vary with model configuration, query shape, filters, and EF Core version.

3. What the sample query means

```
var query = context.Students
    .Include(s => s.Enrollments)
    .ThenInclude(e => e.Course)
    .OrderBy(s => s.StudentId);

Console.WriteLine(query.ToQueryString());
var students = query.ToList();
```

• **ToQueryString()** previews the SQL; it does not execute the query by itself.

• **ToList()** executes the query and materializes results. Runtime logging can show commands actually executed.

• Joined SQL rows may repeat parent columns; EF Core shapes these rows into entities and related collections.

4. Include vs projection

Include / ThenInclude	Select projection
Use when you need entity objects and related navigations, e.g. a student details screen with enrollments and courses.	Use when you need only selected values or summaries, e.g. names, course titles, or enrollment counts. It often avoids loading full entity graphs.

```
var data = context.Students
    .Select(s => new
    {
        s.FirstName,
        Courses = s.Enrollments
            .Select(e => e.Course.Title)
    });
```

For a projection into an anonymous type or DTO, Include() is generally unnecessary: the projection itself defines which data is retrieved.

5. Common mistakes & performance notes

• Use ThenInclude() to continue a path; a collection navigation is not a single entity.

• Avoid loading a huge object graph when the screen needs only a few fields. Excessive Includes can increase data transfer and duplicate joined rows.

• Do not assume Include() always produces exactly one SQL statement; query-splitting settings and query shape can change execution.

• For API responses and dashboards, consider projecting directly into a DTO or a small result shape.

Quick memory anchor

Include = start a relationship path. ThenInclude = continue that path deeper. New Include = another branch.

LEFT JOIN preserves unmatched rows from the left side; **INNER JOIN** keeps only matching rows. Relationship configuration is an important factor, but inspect generated SQL for the exact query.